

Student Name	Rory O'Donoghue	Student Number	21420506
Supervisor	Rosemary Monahan	ECTS Credits	
Project Title	DeductIT		

1) Project Objectives

The project aims to develop an interactive natural-deduction learning game that enhances student engagement through visual proof construction, dynamic animations, and level-based progression. The system will be implemented using React for the frontend and will incorporate animated scopes, structured rule-application slots, and responsive visual cues to support user understanding.

A formally verified foundation is provided by the NaDeA natural-deduction framework, implemented in Isabelle/HOL. The project intends to use Isabelle as the backend theorem checker, enabling the validation of multiple correct proofs beyond the specified solution supplied in the Natural Deduction Pack dataset. A structured database of graded logic exercises will drive gameplay progression.

The core deliverables include: a functional natural-deduction game interface, a proof-validation engine, Isabelle integration, a MySQL-backed data layer, and a gamified progression system.

2) Description of Work Completed

Significant work has been completed across research, system analysis, dataset preparation, and early prototype development. Existing natural-deduction systems were examined to identify limitations such as low visual engagement, restricted feedback, and the absence of gamification. These observations informed key interface and system-design decisions.

A complete dataset of exercises from the Natural Deduction Pack was extracted and converted into structured JSON using AI-assisted Python scripts. Architectural diagrams including flow as shown in Appendix 1, component, and use-case diagrams as shown in Appendix 2, were produced to define system behaviour. Technical analysis of Logiko's internal design provided insights into AST structures, rule checking, and UI behaviours, which guided decisions for the backend logic engine. Planning work identified potential risks related to UI complexity, parsing, and theorem-checking integration.

During initial prototyping, the planned drag-and-drop interaction model was revised. Instead, a structured button-driven insertion mechanism was adopted to ensure consistent placement of proof elements and reduce interaction complexity.

2.1 Evidence of Work Completed

- Literature review and evaluation of existing natural-deduction systems completed as shown in appendix 3
- System block diagram, flow diagram, and component architecture produced.
- Natural Deduction Pack processed into a structured JSON dataset using AI-generated scripts as shown in appendix 4 and appendix 5
- Backend rule-checking approach informed by detailed study of Logiko's parser, rule definitions, and AST.
- UI/UX shortcomings in comparable systems documented to inform visual and interactive design requirements.
- Full architectural specifications for the frontend, backend, logic engine, and database produced.
- Development began and a low-level system was created in which a lot of the key features were included as shown in appendix 6. Those missing were Isabelle integration and visual engagement.

2.2 Literature Review

The literature review focused on three primary areas:

(1) Existing logic tutors and proof assistants: Tools such as Logiko, Pandora, and text-based proof editors provide correctness checking but offer limited animation, visual scaffolding, or gamification. These limitations support the need for a dynamic, interactive learning environment.

(2) Formal verification resources: The NaDeA Isabelle formalisation by Villadsen et al. establishes a verified natural-deduction framework with formally proven soundness and completeness. This framework provides the logical foundation for the project.

(3) Technical integration methods: Research on browser-based theorem-checking approaches, hybrid systems, and backend theorem provers indicates that a browser-based model is currently the most feasible, with future potential for deeper Isabelle server integration.

(4) Academic papers on Isabelle-based teaching frameworks further support the choice of Isabelle as a pedagogical backend for natural-deduction learning.

2.3 Use of GenAI and Tools

GenAI tools contributed effectively in several areas:

- Automated extraction and processing of hundreds of logic exercises into structured JSON via AI-generated and AI-debugged Python scripts.
- Architectural analysis of existing systems assisted by GenAI to clarify component interactions and logic-engine behaviour.
- Generation of preliminary UI concepts and logic-checking pseudocode to support design exploration.

Additional tools used include GitHub Copilot for code assistance and Grammarly for clarity improvements.

3) Future Work

Remaining work includes the full implementation of the React interface, development of the AST-based rule-checking engine, and integration of the JSON exercise dataset into a level-based progression system. Isabelle integration will be completed to validate user-generated proofs beyond predefined solutions. UI improvements such as animation, gamification elements, and dynamic proof-scope visualisations will be added following core functionality.

Planned timeline:

Weeks 1–3: Completion of core functionality including proof-box behaviour and frontend validation based on dataset rules.

Weeks 4–6: Integration of the Isabelle backend to permit multiple-solution validation.

Weeks 7–9: Implementation of interactive and gamified elements including the thematic “witch’s cauldron” proof-assembly metaphor.

Weeks 10–12: Development of backend user features such as saved progress, user profiles, and optional login.

Weeks 12–14: Testing, refinement, accessibility improvements, and validation of full correctness.

User feedback will be collected through surveys of students enrolled in relevant logic modules, facilitated by lecturers teaching natural-deduction topics.

References

Villadsen, J., Jensen, A. B., & Schlichtkrull, A. (2017). NaDeA: A Natural Deduction Assistant with a Formalization in Isabelle. *IFCoLog Journal of Logics and Their Applications*, 4(1), 55–82.

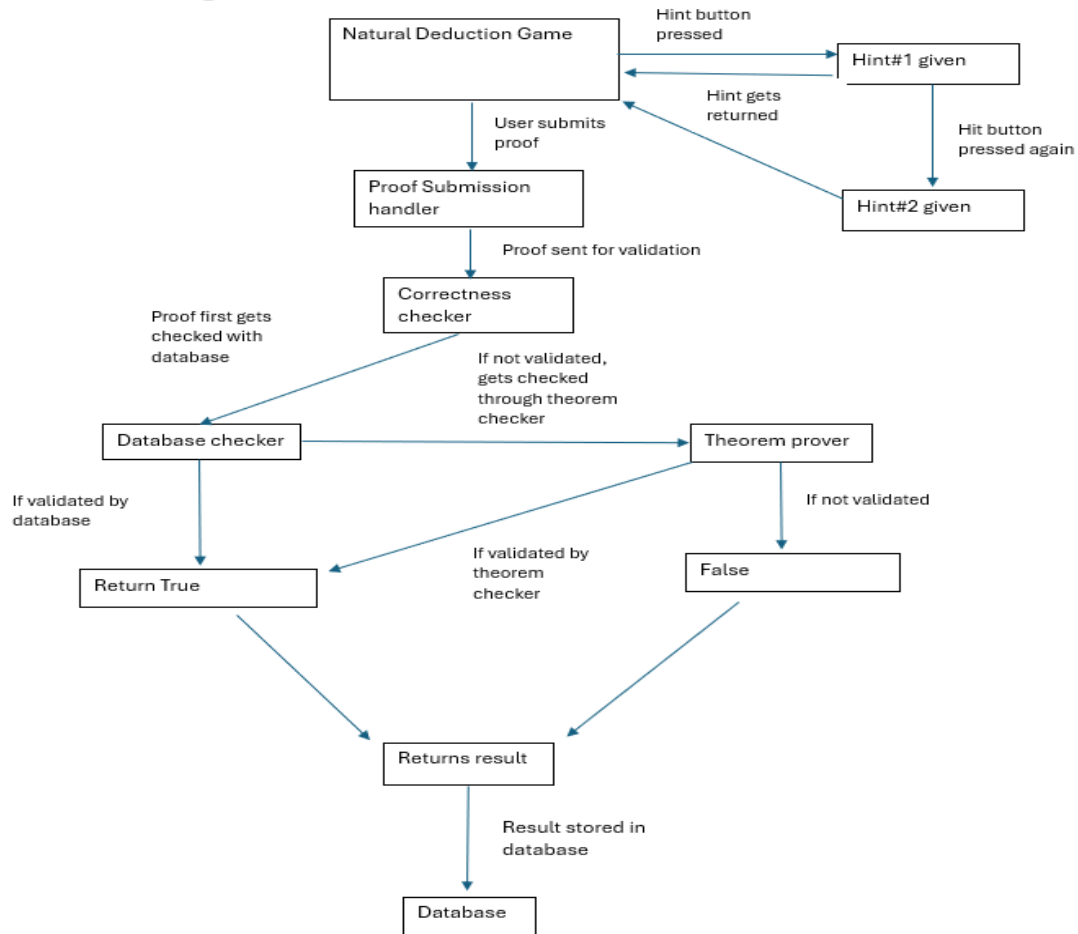
Villadsen, J., From, A. H., & Schlichtkrull, A. (2018). Natural Deduction and the Isabelle Proof Assistant. *EPTCS* 267, 140–155.

From, A. H., Villadsen, J., & Blackburn, P. (2020). Isabelle/HOL as a Meta-Language for Teaching Logic. Electronic Proceedings in Theoretical Computer Science, 1–17.

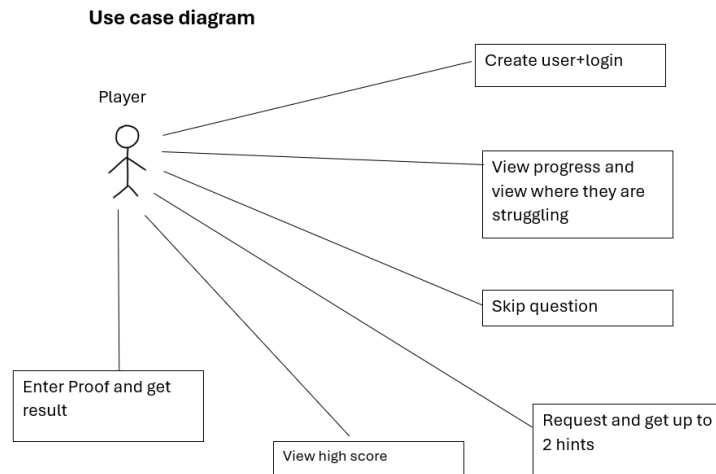
Appendices

Appendix 1

Flow Diagram



Appendix 2



Appendix 3

Tool / Site	Interaction Style	Pedagogy	Visual Engagement	Game Loop	Hint Quality
Natural Deduction (felikso)	Levels	Medium	Low	Basic	Low
Natural Deduction Proof Editor	Editor	Medium	Low	None	Basic
Proof Checker (Open Logic)	Editor	High	Low	None	Medium
Pandora	Guided	High	Low	None	High
Logic Machine	Multi-tool	High	Low	None	Medium
Domino on Acid	Puzzle	Low	Medium	Light	Low
Logiko	Timed Puzzle	Medium	Low	Basic	Low
QED	Narrative	Medium	Low	Unlock	Medium

Appendix 4

```
◆ extract_problems_with_solutions.py 1
C:\Users\royo> royo> OneDrive> Desktop> Final_year> Project> Code> DatabaseExtractionCode> ◆ extract_problems_with_solutions.py > ...
1 import json
2 from PyPDF2 import PdfReader
3
4 input_file = "Natural Deduction Pack.pdf"
5 output_file = "natural_deduction_levels.json"
6
7 reader = PdfReader(input_file)
8 text = ""
9
10 # Skip the first 10 pages [] start reading from page 10 onward
11 for i, page in enumerate(reader.pages):
12     if i < 10:
13         continue
14     page_text = page.extract_text()
15     if page_text:
16         text += page_text + "\n"
17
18 lines = [line.strip() for line in text.split("\n") if line.strip()]
19
20 problems_text = []
21 solutions_text = []
22 collecting_solutions = False
23
24 # Separate problems from solutions
25 for line in lines:
26     if "5 Solutions" in line or "(Solutions" in line:
27         collecting_solutions = True
28         continue
29     if not collecting_solutions:
30         problems_text.append(line)
31     else:
32         solutions_text.append(line)
33
34 # --- Extract problems ---
35 problems = []
36 topic = ""
37 id_counter = 1
38
39 for line in problems_text:
40     # Detect topic headers
41     if any(keyword in line for keyword in [
42         "Core", "Conjunction", "Implication", "Disjunction",
43         "Biconditional", "Negation", "Universal", "Existential", "Identity"]
```

Appendix 5

```
44     ]):
45         topic = line
46         continue
47
48     # Detect numbered problems (like 1. (3))
49     if (
50         (any(char.isdigit() for char in line[:3]) and "(" in line and ")" in line)
51         or line.lower().startswith("problem")
52     ):
53         problems.append({
54             "id": f"ND{id_counter:03}",
55             "topic": topic or "General",
56             "goal": line,
57             "solution": ""
58         })
59         id_counter += 1
60
61 # --- Extract solutions ---
62 solution_index = 0
63 current_solution = []
64
65 for line in solutions_text:
66     if (
67         line.lower().startswith("problem")
68         or (any(char.isdigit() for char in line[:3]) and "(" in line and ")" in line)
69     ):
70         if current_solution and solution_index < len(problems):
71             problems[solution_index]["solution"] = "\n".join(current_solution).strip()
72             current_solution = []
73             solution_index += 1
74         else:
75             current_solution.append(line)
76
77 # Attach last solution if present
78 if current_solution and solution_index < len(problems):
79     problems[solution_index]["solution"] = "\n".join(current_solution).strip()
80
81 # Write to JSON
82 with open(output_file, "w", encoding="utf-8") as f:
83     json.dump(problems, f, indent=2, ensure_ascii=False)
84
85 print(f"Extracted {len(problems)} problems (with possible solutions) to {output_file}")
```

Appendix 6

